

TECHNICAL REPORT: REVERSE ENGINEERING ANALYSIS



UNIVERSIDAD
Escuela Colombiana de Ingeniería Julio Garavito

Project: Lab 05 - Reverse Engineering

Subject: IT Security and Privacy

Professor: Daniel Esteban Vela López

Authors: Andersson David Sánchez Méndez & Cristian Santiago Pedraza Rodríguez

Date: 2026-02-24

Location: Bogotá, Colombia

Confidentiality: Internal / Academic Use Only



1. Introduction

1.1 Objective

The primary objective of this laboratory is to decompose and analyze a provided ELF binary to understand its logical flow and security mechanisms. By combining static and dynamic analysis, we aim to recover hidden data (passwords) and manipulate the program's execution without having the original source code.

1.2 Discussion: The Art of Reverse Engineering

Before the technical execution, we discussed the strategic importance of choosing the right analysis method for security auditing.

Perspective: Andersson David Sánchez Méndez

"From my perspective, the most critical part of this lab is understanding the **Conditional Jumps** (`je` , `jne`). In reverse engineering, these are the decision points. If we can identify where the program compares our input with the hardcoded key, we have essentially bypassed its security. It's not just about finding a string; it's about understanding the logic that guards it."

Perspective: Cristian Santiago Pedraza Rodríguez

"I find the transition from **Static to Dynamic analysis** fascinating. While `objdump` gives us a cold map of the code, `gdb` allows us to see the 'heartbeat' of the program. I noticed that identifying the function prologue and epilogue is vital to not getting lost in the assembly 'sea'. A good report must translate these low-level instructions into a clear business-risk logic."



2. Methodology & Tools

We utilized a multi-layered approach to analyze the binary:

Tool	Purpose	Key Command used
<code>file</code>	Identify architecture and linking	<code>file program</code>
<code>strings</code>	Extract hardcoded passwords/text	<code>`strings program</code>
<code>objdump</code>	Static disassembly	<code>objdump -d program</code>
<code>gdb</code>	Dynamic debugging and register inspection	<code>gdb ./program</code>
<code>Cutter</code>	Visual flowchart and cross-references	<i>GUI Based</i>
<code>radare2</code>	Binary patching (Optional)	<code>radare2 -w ./program</code>



3. PHASE 1: BASIC STATIC INSPECTION

3.1 Binary Information

Based on the execution of the `file program` command, we have identified the fundamental properties of the target file:

- ◆ **Format:** ELF 64-bit LSB pie executable.
- ◆ **Architecture:** x86-64.
- ◆ **Linking:** Dynamically linked.
- ◆ **Debug Symbols:** Not stripped. This is a critical finding as it allows us to see original function names like `main` and `check_pass`, significantly simplifying the reverse engineering process.

3.2 String Discovery

Static analysis using the `strings` utility allowed us to identify critical information vectors before actual execution:

- ◆ **Critical Strings Found:** We detected the authentication string `"ECI_SecuH"` (partially displayed in the grep) and UI messages such as `"Introduce la llave maestra:"`, `"[+] ACCESO CONCEDIDO"`, and `"[-] ERROR: Llave incorrecta"`.
- ◆ **Function Symbols:** We confirmed calls to standard library functions including `strcmp@GLIBC_2.2.5`, `printf@GLIBC_2.2.5`, and `__isoc23_scanf@GLIBC_2.38`. These are essential for the data capture and comparison logic.

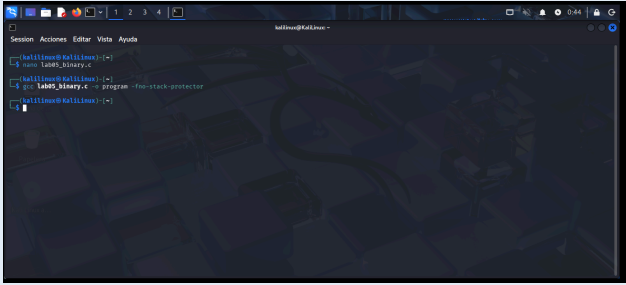
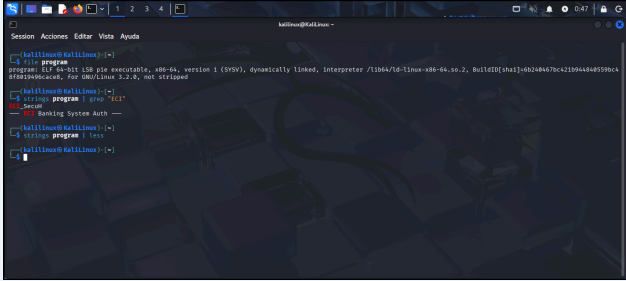
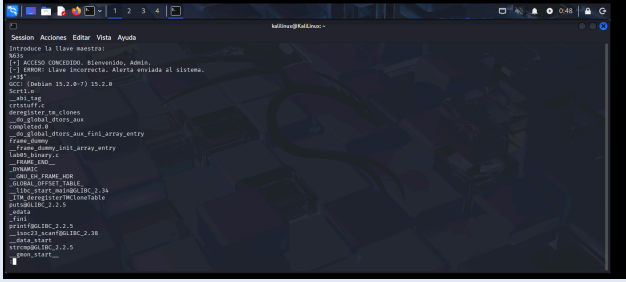
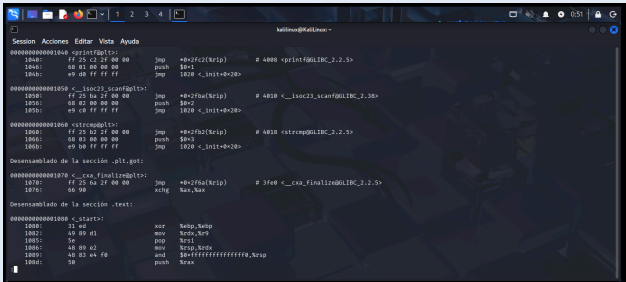
3.3 Flowchart Logic

Through `objdump`, we identified the program's logical structure within the `<main>` function:

1. **Prologue:** Standard stack frame setup using `push rbp` and `mov rbp, rsp`.
2. **Input Phase:** The program calls `printf` to display the prompt and `scanf` to store the user's input at the stack address `[rbp-0x40]`.
3. **Validation Phase:** The program moves the input address into the `rdi` register and executes a `call` to the custom `<check_pass>` function at address `11f9`.
4. **Decision Point:** The return value of the comparison (stored in `eax`) is evaluated using `test eax, eax`. This determines which branch of the code will execute next.



Evidence Annex: Static Analysis

Activity	Command / Evidence	Observation
Binary Creation		Compilation of the <code>lab05_binary.c</code> source code using the <code>-fno-stack-protector</code> flag.
File & Strings		Initial identification of the ELF architecture and discovery of the "ECI" string.
Full Strings List		Detailed inspection of the data section containing system messages and symbols.
Main Function		Disassembly showing the logical flow and the call to the validation function.
Library Calls		Evidence of external library linking for <code>strcmp</code> and <code>printf</code> .



4. PHASE 2: DYNAMIC DEBUGGING WITH GDB

4.1 Step-by-Step Analysis

In this phase, we moved from static observation to dynamic execution using **GDB (GNU Debugger)**. This allowed us to monitor the program's state in memory and its interaction with CPU registers during the authentication process.

- 1. Initialization and Interception:** The debugger was launched with the target binary, and a breakpoint was successfully set at the `main` function (address `0x11b4`). This paused execution before the first instruction of the entry point was processed.
- 2. Execution and Input Tracing:** We utilized the `run` command to reach the breakpoint and `next` to step through the execution. Upon reaching the input prompt, the "Master Key" (123456) was provided, resulting in an immediate "ERROR" message as the execution overshot the comparison logic due to the lack of source line information.
- 3. State Inspection:** After the program processed the input, `info registers` was executed to capture the final state. We observed the `RAX` register at `0x0` and the `RIP` (Instruction Pointer) at `0x7ffff7c29f68`, indicating the process was handling exit routines within the C library.

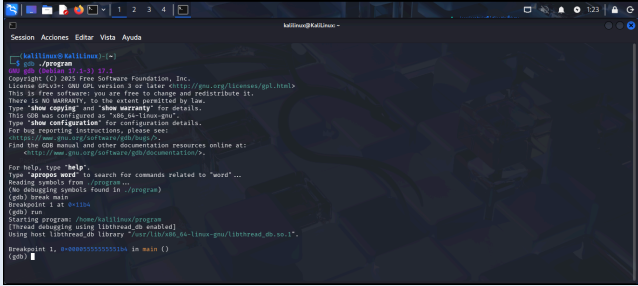
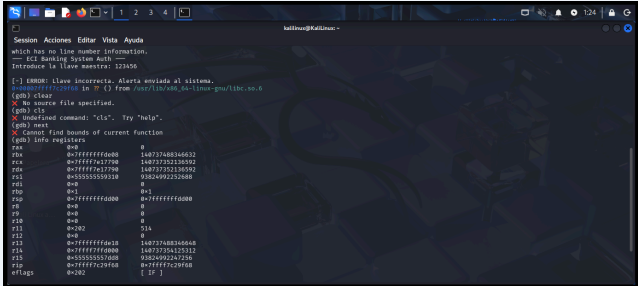
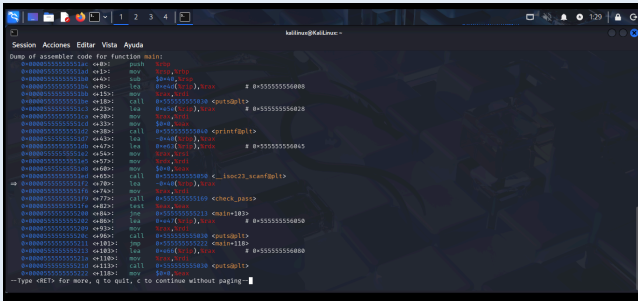
4.2 Code Disassembly and Decision Identification

To pinpoint the exact location of the security check, we executed `disas main`, which revealed the underlying assembly structure:

- ◆ **Input Capture:** The program calls `__isoc23_scanf@plt` at offset `+65` to store the user's input in the local stack buffer at `[rbp-0x40]`.
- ◆ **Logical Comparison:** The authentication logic is delegated to the custom function `<check_pass>`, called at offset `+77`.
- ◆ **The Critical Jump:** The most vital instruction identified is the `test eax, eax` followed by `jne <main+103>` at offset `+82`. This conditional jump acts as the "gatekeeper," deciding whether to grant access or trigger the error alert based on the comparison result.



Evidence Annex: Dynamic Analysis

Activity	Command / Evidence	Technical Observation
Breakpoint & Run		Confirmation of entry point at 0x11b4 and debugger synchronization.
Register Dump		Capture of CPU state and stack pointer status after input processing.
Main Disassembly		Identification of the check_pass call and the conditional branching logic.



5. PHASE 3: VISUAL ANALYSIS (CUTTER)

5.1 Graphical Logic Flow

The visual analysis performed with **Cutter** allowed for the synthesis of previous findings by representing the binary's execution as a directed control flow graph. This perspective facilitates the identification of "Basic Blocks" and the program's decision-making process regarding security.

- ◆ **Environment Initialization:** The tool was launched using the command **cutter ./program**. The terminal output confirmed that Cutter performed an automatic analysis of functions, local variables, and DWARF information, identifying the binary as an x86-64 Linux executable that was not stripped.

- ◆ **Code Decomposition:** Within the `main` function, Cutter's disassembly view clearly labeled the custom function `<sym.check_pass>` and the stack offsets for local variables. We identified the function prologue and the allocation of stack space via `sub rsp, 0x30` to prepare the validation environment.
- ◆ **The Logical "Fork":** The Graph View provided the definitive evidence of the program's conditional logic. Following the `call sym.check_pass` and `test eax, eax` instructions, the flow reaches a conditional jump (`jne 0x1213`) that splits the execution into two critical paths.

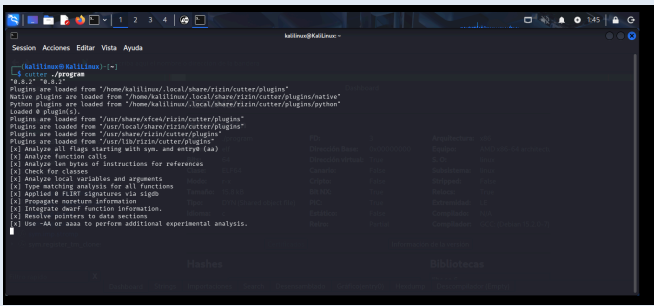
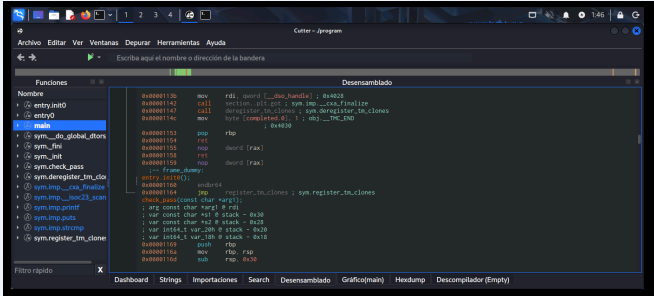
5.2 Branch Identification

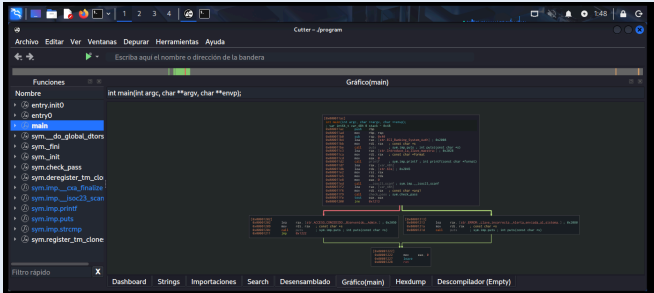
The visual graph uses color-coded edges (arrows) to indicate the result of the password comparison:

- ◆ **Success Path (Green Arrow):** Leads to the left block at address `0x00001202` , which loads the string `[+] ACCESO CONCEDIDO. Bienvenido, Admin.` into memory for display.
- ◆ **Failure Path (Red Arrow):** Leads to the right block at address `0x00001213` , which loads the error string `[-] ERROR: Llave incorrecta. Alerta enviada al sistema.` .



Evidence Annex: Visual Analysis

Activity	Command / Evidence	Technical Observation
Cutter Launch		Initial analysis phase identifying architecture and binary symbols.
Main Disassembly		Detailed instruction view showing the <code>check_pass</code> call and stack setup.

Activity	Command / Evidence	Technical Observation
Graph Logic		Full visual map of the authentication branches (Success vs. Failure).



6. PHASE 4: BINARY MANIPULATION WITH RADARE2 (OPTIONAL)

6.1 Logic Patching Methodology

In this final phase, we performed direct binary manipulation using **radare2** to attempt a bypass of the authentication mechanism. The goal was to modify the machine code to ignore or invert the password validation result.

- Opening and Initial Analysis:** We launched the tool in write mode using `radare2 -w ./program`. We then executed `aaa` for a comprehensive analysis of symbols and `s main` to navigate to the entry point of the primary logic.
- Mapping the Function:** By executing the `pdf` (Print Disassembly Function) command, we obtained the full assembly listing for `main`. This allowed us to confirm the exact memory address of the critical decision point.
- Instruction Modification:**
 - ◆ We navigated to the address of the jump instruction: `s 0x0001200`.
 - ◆ We attempted to rewrite the instruction using `wa je` (to invert the logic) and also tested a NOP-out strategy using `wa nop; wa nop` to neutralize the jump entirely.

6.2 Verification and System Response

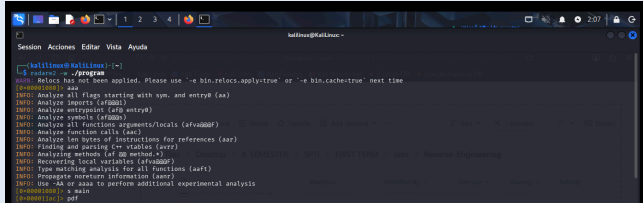
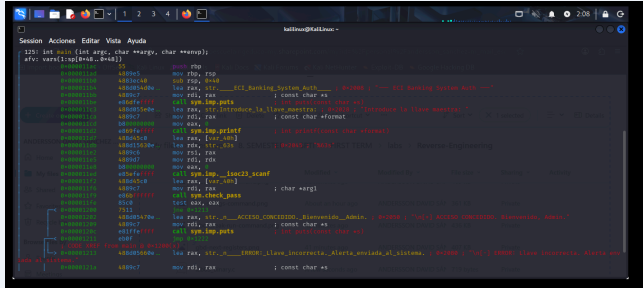
The verification process highlighted the sensitivity of binary instruction alignment:

- ◆ **Access Attempt:** After applying the patch, we exited radare2 and executed `./program`, providing the arbitrary key "hacked".
- ◆ **Technical Outcome:** The program triggered a `zsh: segmentation fault`.
- ◆ **Root Cause Analysis:** As seen in the terminal trace, the original `jne` instruction occupied 6 bytes. Replacing it with `NOP` instructions of different total lengths or a

misaligned **JE** opcode caused the CPU to misinterpret the subsequent bytes as invalid instructions, leading to a memory access violation (crash).



Evidence Annex: Binary Patching

Activity	Command / Evidence	Technical Observation
Analysis & Entry		Load binary in write mode and locate the main symbol.
Function Map		Identification of the conditional jump at address 0x00001200 .
Logic Edit		Written 6 bytes to modify/neutralize the jump instruction.
Verification		Execution failed with a segmentation fault due to instruction misalignment.



7. CONCLUSIONS

The analysis performed in this laboratory leads to the following streamlined conclusions:

- ◆ **Fragility of Client-Side Security:** This lab proves that "Security by Obscurity" is ineffective. Static analysis tools like **strings** easily expose hardcoded secrets, demonstrating that sensitive validation should never rely solely on local binary logic.

- ◆ **Methodological Synergy:** Success in reverse engineering requires a hybrid approach. While `objdump` and `Cutter` provide the structural "map," `gdb` is essential to observe the "heartbeat" of the program—specifically how registers like `RAX` handle authentication results in real-time.
- ◆ **Precision of Binary Patching:** The `Segmentation Fault` encountered during the `radare2` phase highlights that patching is not just about logic, but about **architectural precision**. Misaligning a single byte can crash a program, proving that binary manipulation requires exact opcode sizing.
- ◆ **Visibility of Symbols:** The presence of function names (non-stripped binary) significantly accelerated our analysis. This underscores why developers use symbol stripping and obfuscation as primary layers of defense against unauthorized code analysis.



8. REFERENCES

- ◆ **Eilam, E.** (2011). *Reversing: Secrets of Reverse Engineering*. Wiley Publishing.
- ◆ **Radare2 Project.** (2025). *Radare2 Book: Official Documentation*. <https://book.rada.re/> 
- ◆ **Cutter Project.** (2025). *Cutter: Visual Reverse Engineering Platform*. <https://cutter.re/> 
- ◆ **GDB Project.** (2025). *GNU Project Debugger Documentation*. <https://www.gnu.org/software/gdb/> 
- ◆ **Intel Corporation.** (2024). *Intel® 64 and IA-32 Architectures Software Developer Manuals*.
- ◆ **IEEE Computer Society.** *Ethics of Reverse Engineering for Security Research*.

